

PrecisionRAG: Hybrid, Reranked Vector Retrieval for Hallucination-Resistant RAG

Vector Database Design for Large-Scale Precision Retrieval in RAG Systems

Team **eyaaduhcaam**

Himanshu · Shubham · Priyanshu · Prathamesh

BIT Mesra

01 Problem Understanding

RAG systems are only as good as what they retrieve. The LLM will ground its answer on whatever reaches its context window, so a chunk that is *semantically close but factually wrong* becomes a confident hallucination. Pure dense vector search makes this worse at scale, for three reasons:

- **Lexical blindness:** embeddings blur exact tokens such as names, numbers, codes and acronyms. "Form 1099-B" and "Form 1099-R" land near each other, but only one is correct.
- **Ranking noise:** bi-encoders score the query and the passage independently, so the top 5 often contains near-miss passages that only a query-aware model can separate out.
- **Operational gaps:** enterprise use needs scoped search (a filter by source or category), updates without re-indexing, and measurable quality. Prototypes usually skip all three.

Our goal: a retrieval layer that runs on a CPU, uses only free tools, and **measurably beats** a naive dense baseline, targeting RAGAS **Context Precision > 0.75**, **Context Recall > 0.70** and **p95 latency < 300 ms** over **500K MS MARCO passages** (the bonus scale). Every claim will be backed by logged evaluation runs.

Requirement mapping

Requirement	How we meet it
FR-1 Ingestion	HF <code>datasets</code> streaming of MS MARCO v2.1 → dedupe → batch-embed (BGE-small, ONNX int8) → Qdrant with payload {pid, source domain, category}. Budget: < 2 h on CPU for 500K passages.
FR-2 Dense baseline	Cosine HNSW search, top 5, latency logged per query, RAGAS on 50 queries (the spec asks for ≥ 20).
FR-3 Hybrid	BM25 sparse vectors in the <i>same</i> collection. Server-side RRF (k = 60) or weighted-linear fusion (α configurable in YAML), both documented.
FR-4 Filtering	Payload indexes on <code>category</code> and <code>source</code> . Qdrant applies the filter <i>inside</i> the HNSW traversal, so it is genuinely pre-retrieval.
FR-5 Live updates	REST POST <code>/passages</code> (upsert: embed + write dense and sparse) and DELETE <code>/passages/{id}</code> . Demonstrated live in the UI.
FR-6 Interface	Streamlit UI with a Dense / Hybrid / Hybrid + Rerank toggle, per-passage scores, a stage-by-stage latency breakdown, filter controls, and a RAGAS dashboard tab.

02 Proposed Approach

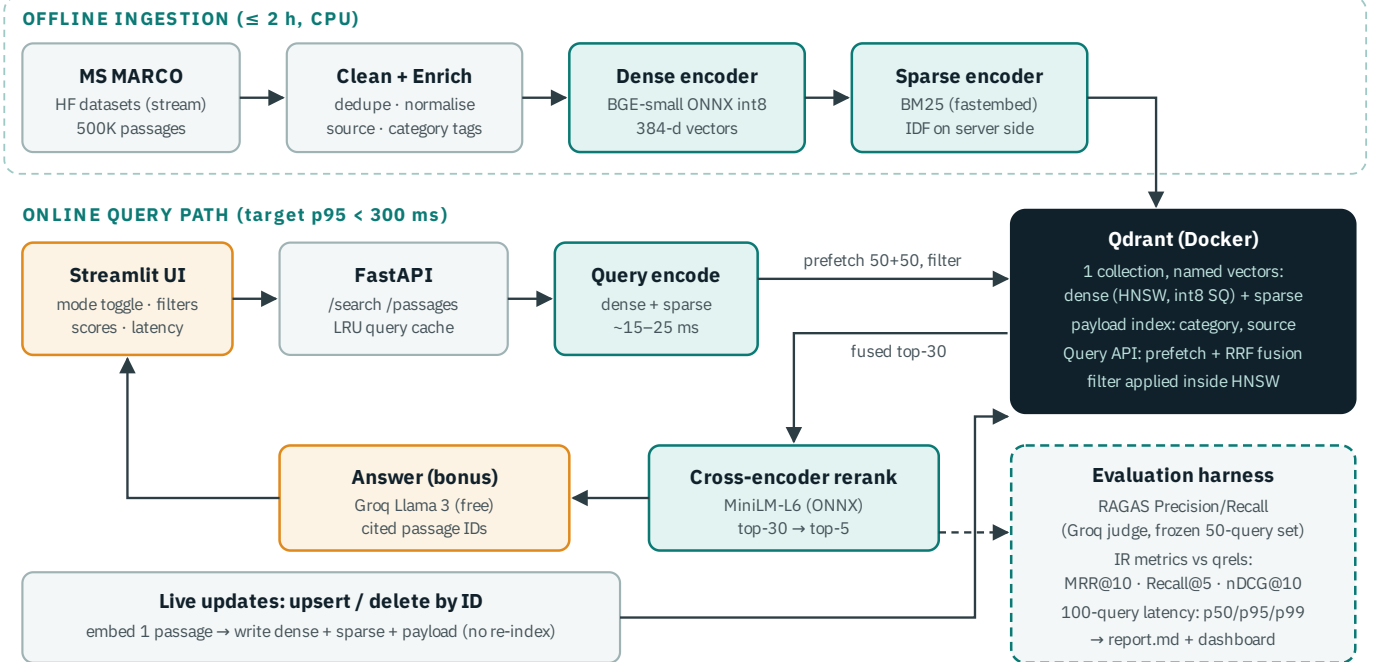
Phase 1: Clean baseline (build it once, build it right)

- Stream 500K passages. Normalise whitespace, drop near-duplicates by hash, and keep passages under 512 tokens (MS MARCO passages are already short).
- **Embeddings:** BAAI/bge-small-en-v1.5 (384-d), exported to ONNX with int8 dynamic quantization for about 2–3× faster CPU encoding. Batch size 256, multiprocessing.
- **Index:** Qdrant HNSW (m = 16, ef_construct = 128), int8 scalar quantization with original vectors kept on disk for rescoring. Memory drops about 4× with negligible recall loss.
- Log RAGAS Context Precision and Recall as the **baseline**, with the same query set frozen for all later phases.

Phase 2: Precision stack

- **Hybrid recall:** dense top 50 plus BM25 top 50, fused with RRF in *one* Qdrant Query API call, so there is no second engine to keep in sync.
- **Cross-encoder rerank:** ms-marco-MiniLM-L-6-v2 rescoring the fused top 30 → top 5. This is the largest precision gain because it reads the query and passage *together*.
- **Metadata:** `source` = URL domain (native to MS MARCO); `category` = query-type taxonomy (numeric, entity, location, person, description) propagated to passages.
- **Speed:** LRU cache of query embeddings and results, warm-loaded models, and a tuned `hnsw_ef`.
- **Answering (bonus):** Groq free tier (Llama 3) generates the answer with **inline citations** to passage IDs.

03 Solution Architecture



Why these design choices

Decision	Rationale and trade-off
Qdrant over Chroma / pgvector	Native sparse vectors <i>and</i> server-side fusion in one query, true filtered HNSW (no post-filter recall collapse), scalar quantization, and a single Docker container. Chroma lacks native hybrid search, and pgvector needs a separate full-text query and manual fusion.
RRF (default) vs weighted-linear	RRF is scale-free: dense cosine and BM25 scores live on different ranges, so fusing by rank is robust without calibration. Weighted-linear (min-max normalised, α tunable) is kept as a switch, and we will report both.
Retrieve 50 → rerank 30 → return 5	A wide first stage protects <i>recall</i> , and the cross-encoder buys <i>precision</i> . Top-30 reranking on a CPU with ONNX is budgeted at about 120–180 ms. If p95 slips, we lower the candidate count to 20 and report the precision/latency curve.
int8 quantization (embeddings + index)	Makes 500K-scale indexing feasible within 2 h on a laptop CPU and keeps the index in RAM. Qdrant rescoring with the original vectors limits the accuracy loss.

Latency budget (per query, CPU)

Stage	Encode (dense + sparse)	Qdrant hybrid + filter	Rerank top-30	API overhead	Total budget
Target	≤ 25 ms	≤ 40 ms	≤ 180 ms	≤ 15 ms	≤ 260 ms p95 (cache hit: < 5 ms)

04 Technology Stack (100% free tier)

Layer	Choice	Why
Data	HuggingFace <code>datasets</code> (MS MARCO v2.1)	Streaming loader; qrels available for ground-truth IR metrics.
Dense embeddings	BAAI/ <code>bge-small-en-v1.5</code> via ONNX Runtime / fastembed	Strong retrieval quality for its size; fast on a CPU.
Sparse / keyword	BM25 sparse vectors (fastembed <code>Qdrant/bm25</code>)	Exact-term matching stored next to the dense vectors.
Vector database	Qdrant (Docker, v1.10+ Query API)	Hybrid fusion, filtered HNSW, quantization, upsert/delete.
Reranker	<code>cross-encoder/ms-marco-MiniLM-L-6-v2</code>	Trained on MS MARCO, so it matches the domain; 22M parameters, CPU-friendly.
Backend / UI	FastAPI + Pydantic · Streamlit	Typed REST API for updates; fast interactive demo.
LLM / Judge	Groq free tier (Llama 3.x)	Cited answers and the RAGAS judge, with no paid APIs.
Evaluation	RAGAS · <code>ranx/pytrec_eval</code> · custom latency bench	LLM-judged and human-labelled metrics that cross-check each other.

05

Evaluation Plan: proving the improvement

- **Frozen query set:** 50 MS MARCO dev queries with human-labelled relevant passages (qrels), used unchanged for every configuration.

• **Ablation ladder:** ① dense → ② + BM25 hybrid (RRF) → ③ + cross-encoder rerank → ④ + filters. Each step's gain is reported separately, so judges can see *where* the precision comes from.

• **Two lenses:** RAGAS Context Precision and Recall (LLM judge) *and* MRR@10 / Recall@5 against qrels. Because the metrics agree, the judge's scoring cannot be the reason for any improvement.
- **Latency:** 100 consecutive, cache-disabled queries → p50/p95/p99 per stage, plotted as a histogram.

• **Scale evidence:** indexing throughput (passages/s), total time, RAM and disk footprint at 100K vs 500K.

• **Deliverables:** `benchmark_report.md/pdf` (Phase 1 vs Phase 2), raw JSON logs, architecture diagram, and a README with reproduction steps.

Metric	Phase 1 (dense)	Phase 2 target	Acceptance (NFR)
RAGAS Context Precision	measured baseline	≥ 0.80	> 0.75
RAGAS Context Recall	measured baseline	≥ 0.75	> 0.70
p95 latency (100 queries)	logged	< 260 ms	< 300 ms
Passages indexed	100K+	500K	100K min · 500K bonus

06

24-Hour Execution Plan and Risk Mitigation

Hours	Milestone	Risk	Mitigation
Pre-event	Repo skeleton, Docker Compose, embeddings pre-computed and cached, query set frozen.	500K indexing exceeds 2 h	ONNX int8 + batching; checkpointed resumable ingest; 100K always ready as a fallback.
0–4	Phase 1 ingestion (100K) + dense search + UI → baseline RAGAS logged .	Rerank breaks p95	Tune candidate count (30 → 20); cache; report the precision/latency trade-off honestly.
4–10	Sparse vectors, RRF hybrid, payload filters, upsert/delete API.	Groq rate limits in RAGAS	Throttled async calls, cached judge results, IR metrics as an independent check.
10–15	Cross-encoder rerank, caching, scale to 500K, latency tuning.	Sparse category labels	Taxonomy from query type + URL domain, both deterministic and documented.
15–20	Full evaluation + ablations, Groq cited answers, dashboard.		
20–24	Benchmark report, README, demo rehearsal, buffer.		

What judges will see on demo day: the same question answered in Dense vs Hybrid vs Hybrid + Rerank modes side by side, with per-passage scores and stage latency; a filtered query; a passage upserted and deleted live; and a dashboard showing the measured Phase 1 → Phase 2 RAGAS gain.